大模型微调

基础概念+手撕代码+操作指南

LLM实战速成班

大模型微调

第5课-微调.pptx	09/02/25 下午 5:40	2.1 MB	PowerP... (.pptx)
概念代码	今天 下午 3:43	--	文件夹
概念代码.zip	今天 下午 3:42	10 KB	ZIP 归档
deepseek-r1-Distilled Llama 8B医疗推理模型微调实战	今天 下午 3:11	--	文件夹
FreedomIntelligence/medical-o1-reasoning-SFT	今天 上午 2:34	--	文件夹
FreedomIntelligence/medical-o1-reasoning-SFT.zip	今天 上午 2:37	24.6 MB	ZIP 归档
qwen2.5法律大模型实战	今天 下午 3:09	--	文件夹
DISC-Law-SFT-Triplet-released.jsonl	今天 上午 1:15	52.8 MB	JSONL
LLaMA-Factory	今天 上午 8:09	--	文件夹
process_data.py	今天 上午 8:41	1 KB	Python Script
Qwen/Qwen2.5-3B-Instruct	今天 上午 12:41	--	文件夹
Qwen2.5法律大模型微调手册.pdf	今天 下午 2:18	1.1 MB	PDF 文稿
unsloth/DeepSeek-R1-Distill-Llama-8B	今天 上午 2:17	--	文件夹
unsloth/DeepSeek-R1-Distill-Llama-8B.zip	今天 上午 2:38	12.72 GB	ZIP 归档

<think>
这个病人已经有五天的急性阑尾炎了，虽然他的腹痛已经稍微减轻了，但他仍然发烧，还在右下腹部有一个压痛的包块。嗯，这种情况下我觉得最重要的是要控制他的发烧。

首先，我想到的就是使用退烧药。退烧药可以很快地将他的体温降下来，这样可以缓解他的不适，减轻他的疼痛感。

然后，我想到抗生素。因为急性阑尾炎通常是由于细菌感染引起的，抗生素可以帮助清除这些细菌，减少感染的风险，保护他的阑尾。

接下来，我想到肠道保护剂。虽然他现在可能不需要立即进行手术，但肠道保护剂可以帮助他的肠道黏膜恢复，预防感染扩散，确保他的肠道健康。

最后，我想到如果包块已经被诊断为炎症性包块，手术就成了一个重要的选择。手术不仅能彻底清除病变，还能防止未来的感染。

所以，综合考虑，我觉得在这种情况下，应该首先使用退烧药来缓解发烧，然后用抗生素清除感染，接着用肠道保护剂保护肠道，最后根据情况进行手术处理。这样可以全面地保护他的健康。
</think>
对于一个患有急性阑尾炎且腹痛稍有减轻但仍然发热的病人，在右下腹部有压痛的包块时，应采取以下步骤进行处理：

1. **退烧药治疗**：首先应使用退烧药以迅速降低病人的发烧，这可以缓解其不适和疼痛感。

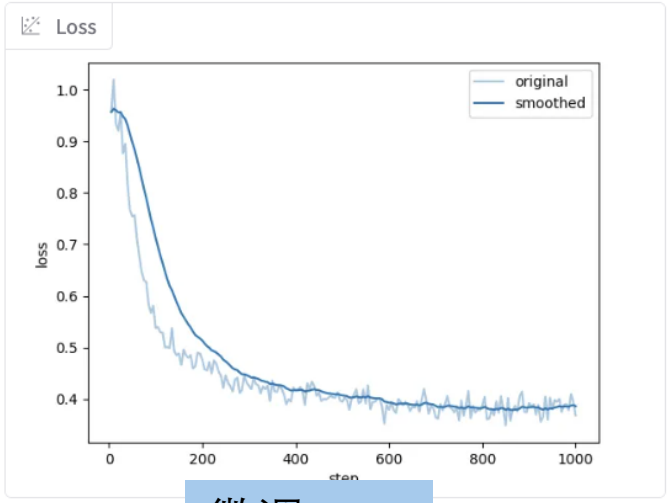
2. **抗生素治疗**：由于急性阑尾炎通常由细菌感染引起，抗生素可以帮助清除感染，降低感染风险，并保护阑尾。

3. **肠道保护剂**：虽然病人可能不需要立即手术，但肠道保护剂可以帮助恢复肠道黏膜，预防感染扩散，维护肠道健康。

4. **手术处理**：如果包块已被诊断为炎症性包块，应考虑进行手术以彻底清除病变，防止未来的感染。

综上所述，首先应使用退烧药缓解发烧，接着使用抗生素清除感染，使用肠道保护剂保护肠道，最后根据病情进行手术处理。<｜end_of_sentence｜>

微调dsr1-llama



微调qwen

1

微调

预训练之后为什么还要微调?

1.1 定义

Base Model (Llama3.1 8b base)

Prompt: Tell me how to fine-tune a model

Completion:

- How can I control the complexity of a model?
- How do I know when my model is done?
- How do I test a model?
- How do I define a task for my model?
- How do I scale a model?
- How do I make a probabilistic model?
- How do I start using a model once it's complete?
- How do I cope with missing data?
- How do I make my modeling efforts sustainable?

Base Model (Llama3.1 8b instruct)

Prompt: Tell me how to fine-tune a model

Completion: Fine-tuning a model involves adjusting the parameters of a pre-trained model in order to make it better suited for a given task. There are generally three steps to fine-tuning a model:

1. **Select a base model** – Select a pre-trained deep learning model that has been trained on a large dataset.
2. **Adjust parameters** – Adjust parameters of the pre-trained model to better suit the desired task. This may include changing the number of layers, adjusting learning rate, adding regularization, or tweaking the optimizer.
3. **Train the model** – Train the new model on the desired dataset. The amount of data and the amount of training required will depend on the task and the model.

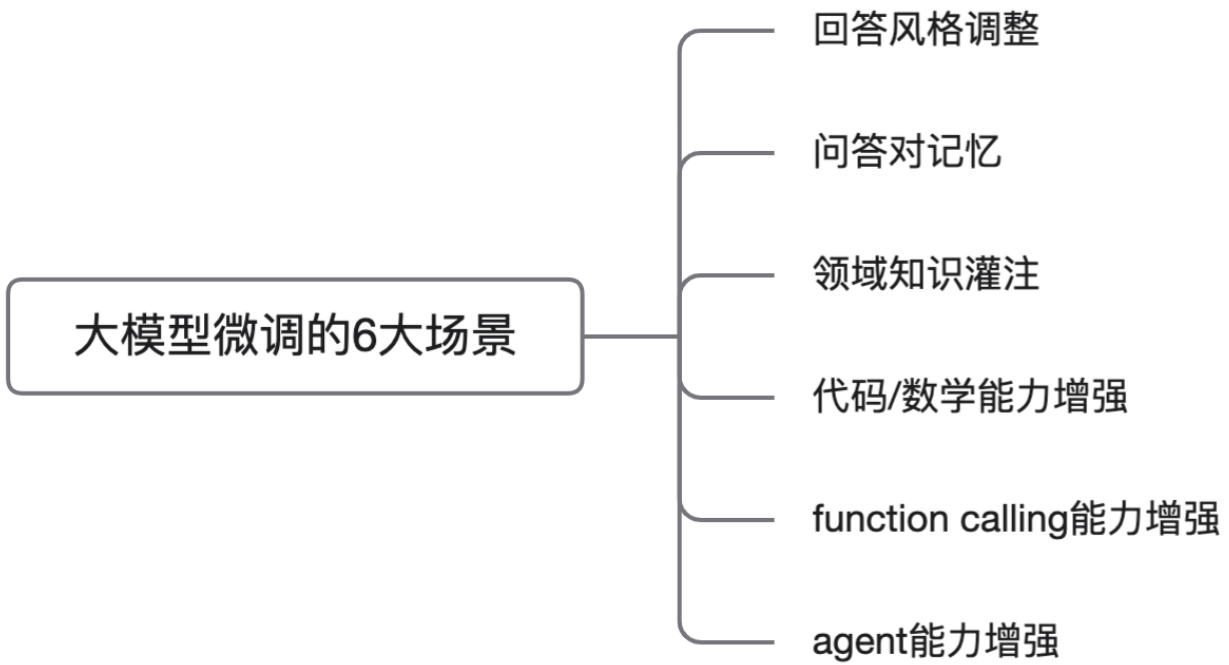
1.1 定义

- **目标：**

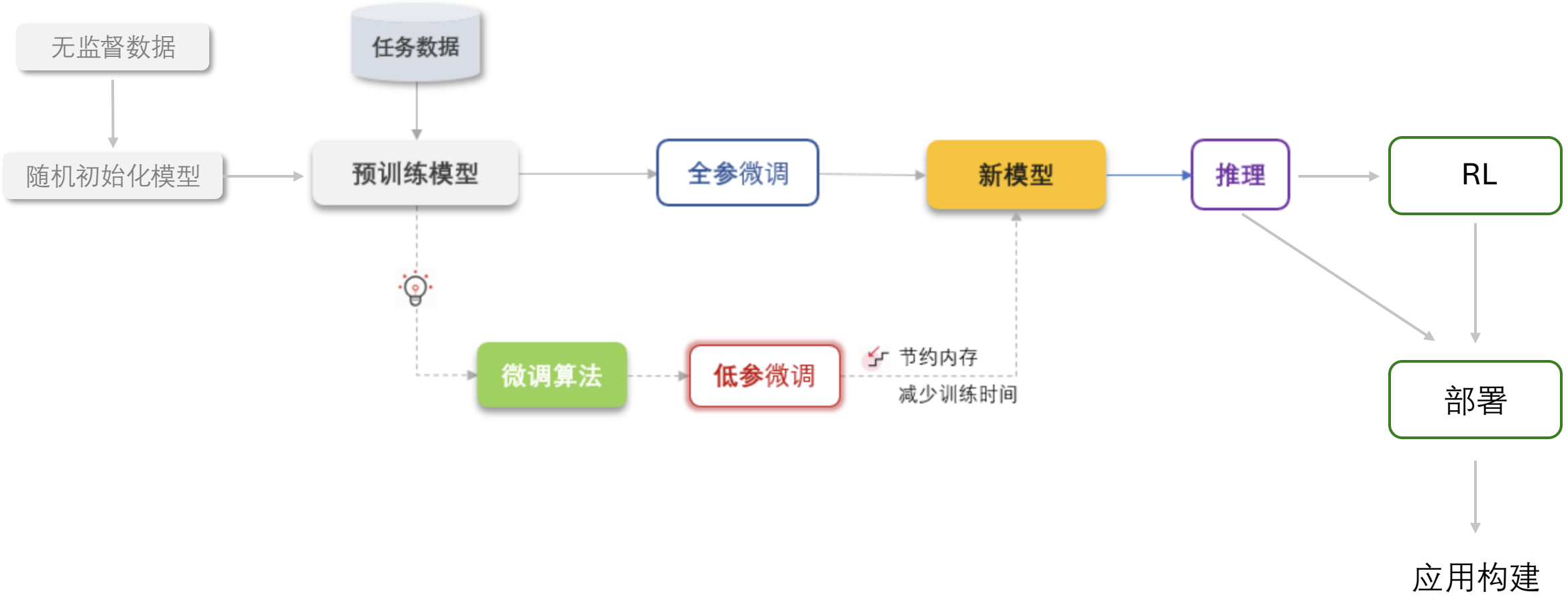
  在预训练阶段，模型在大规模标注文本上学习对下一个token的分布表示；**在下游应用中，通过微调（fine-tuning）可使模型适应特定任务领域或风格（如问答、代码生成、对话系统等）。最核心的是学会指令遵循，把知识按照要求表达出来**

- **微调的重要性**

  大型模型训练通常只学习语言分布和常识知识，但实际任务往往需要更精确的指令与特定的输出格式。微调能够使预训练模型在特定任务的分布上进行训练，通过少量任务数据输入与反馈调整模型输出质量，进而在生成质量与可控性上获得更大提升。



1.1 定义


无监督数据
任务数据
随机初始化模型
预训练模型
全参微调
新模型
推理
RL
微调算法
低参微调
节约内存
减少训练时间
部署
应用构建

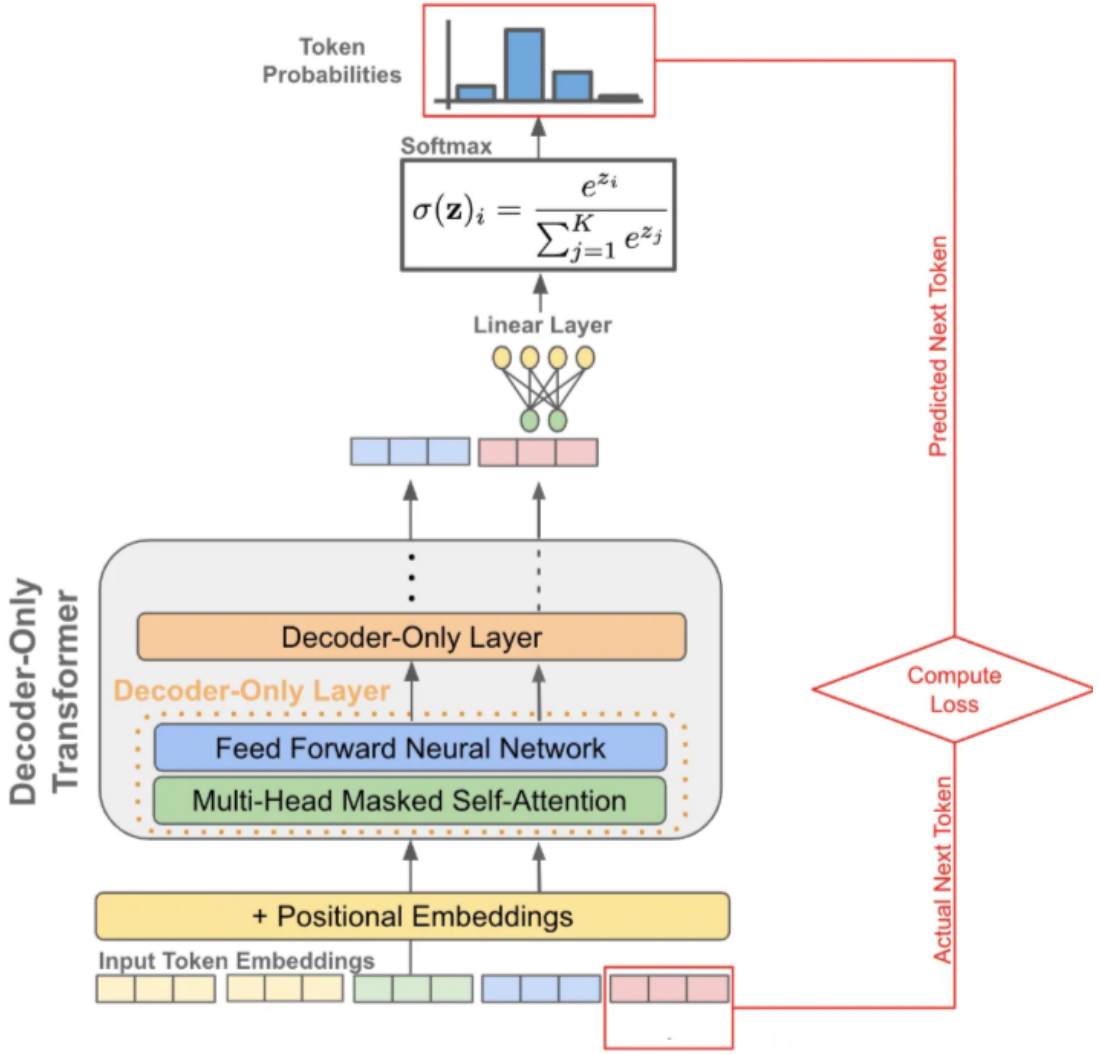
1.2 微调和预训练有什么区别

大模型微调

1.Chat Template　　2.Completions only　　3.NEFTune

SFTTrainer

1.2.1chat template



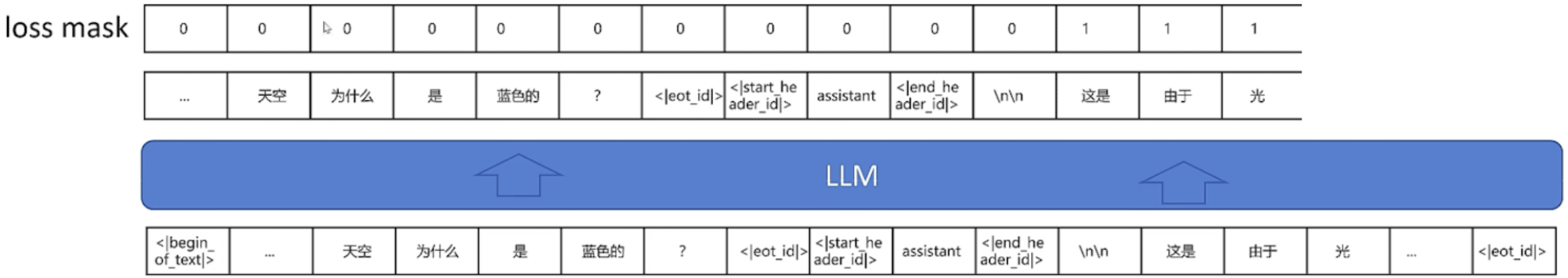
看上去和预训练计划完全一样，那么区别在哪里？

模型结构？数据？Loss计算？

[{ "role": "system", "content": "You are a helpful assistant." },
{ "role": "user", "content": "天空为什么是蓝色的?" },
{ "role": "assistant", "content": "这是由于光的散射引起的。" }]

<begin_of_text>
<start_header_id>system<end_header_id>
You are a helpful assistant.<eot_id><start_header_id>user<end_header_id>
天空为什么是蓝色的?
<eot_id><start_header_id>assistant<end_header_id>
这是由于光的散射引起的。<eot_id>

1.2.2 completion only



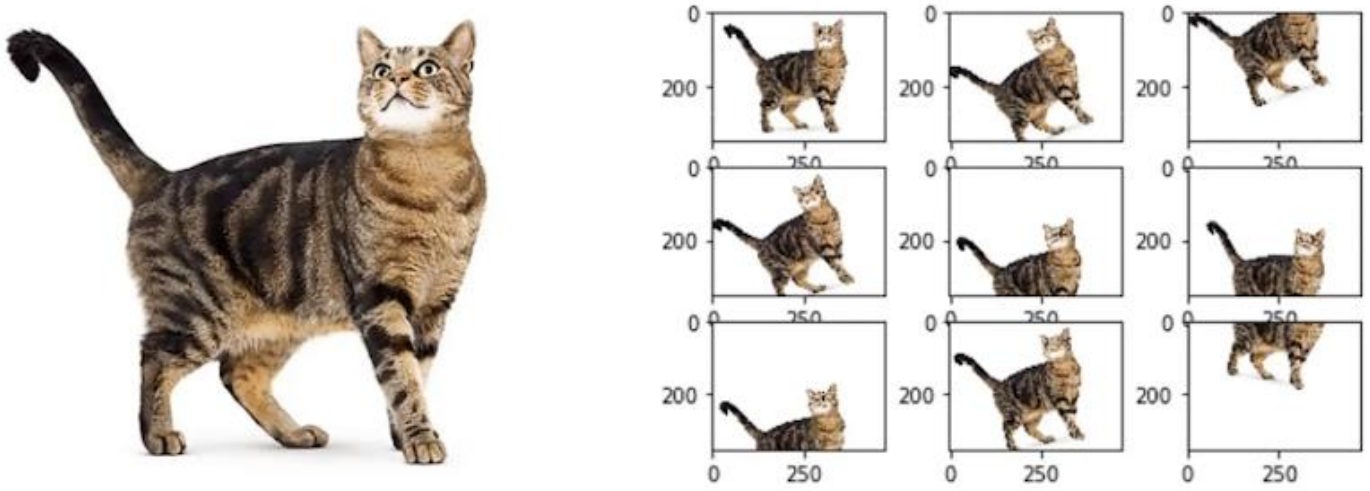
<begin_of_text> <start_header_id>system<end_header_id>
You are a helpful assistant.<eot_id><start_header_id>user<end_header_id>
天空为什么是蓝色的? <eot_id><start_header_id>assistant<end_header_id>

大模型输出:

这是由于光的散射引起的。<eot_id>

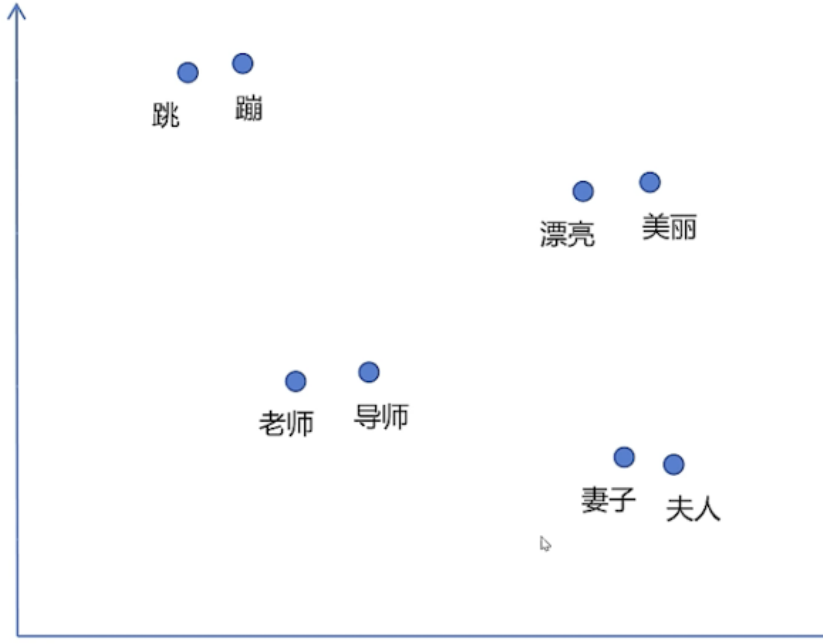
只有这里计算loss

1.2.3 NEFTune



图像领域的数据增强

1.2.3 NEFTune



Word Embedding

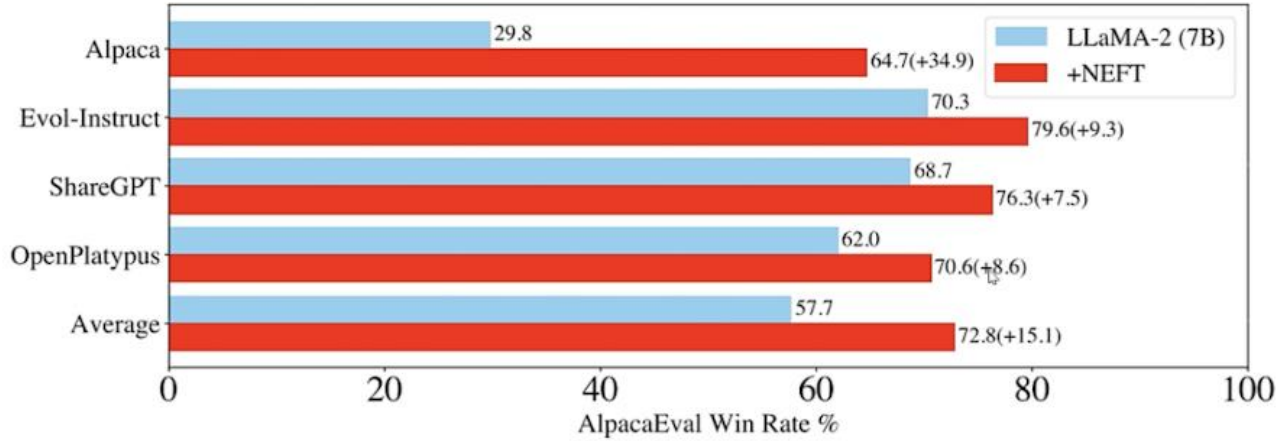


Figure 1: `AlpacaEval` Win Rate percentage for LLaMA-2-7B models finetuned on various datasets with and without `NEFTune`. `NEFTune` leads to massive performance boosts across all of these datasets, showcasing the increased conversational quality of the generated answers.

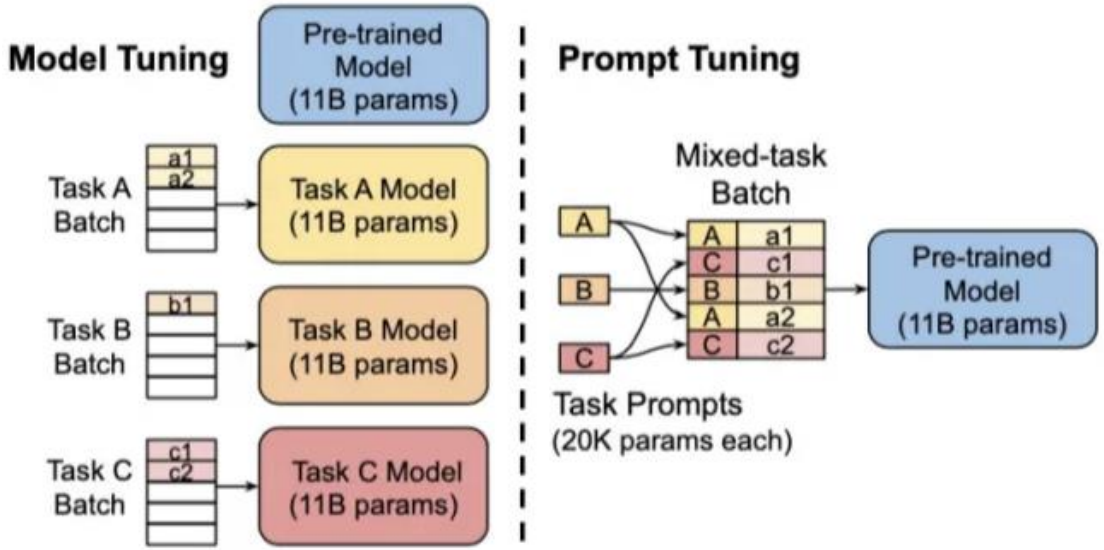
```
neftune_noise_alpha = 10
for i in range(epoch):
    for inputs, loss_mask in data_loader:
        input_ids = inputs.pop("input_ids")
        input_embeddings = model.base_model.model.model.embed_tokens(input_ids)
        dims = torch.tensor(input_embeddings.size(1) * input_embeddings.size(2))
        mag_norm = neftune_noise_alpha / torch.sqrt(dims)
        input_embeddings = input_embeddings + torch.zeros_like(input_embeddings).uniform_(-mag_norm, mag_norm)
        inputs["inputs_embeds"] = input_embeddings
```

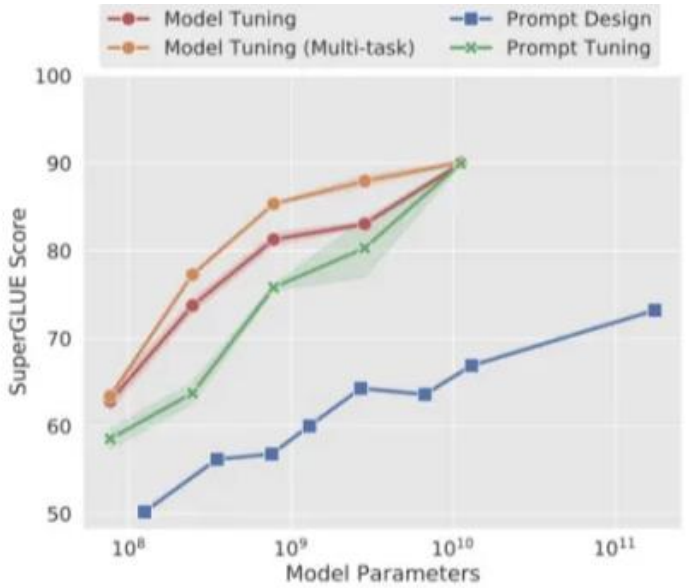
1.3 如何微调

- 1.3.1 全量微调
- 1.3.2 prompt-tuning
- 1.3.3 p-tuning
- 1.3.4 prefix-tuning
- 1.3.5 adapter
- 1.3.6 LoRA

1.3.2 prompt-tuning

- **Prompt-Tuning的思想**：流结主模型全部参数，在训练数据前加入一个小段Prompt，只训练Prompt的表示层，即一个Embedding模块。其中，Prompt又存在两种形式，一种是hard prompt，一种是soft prompt。



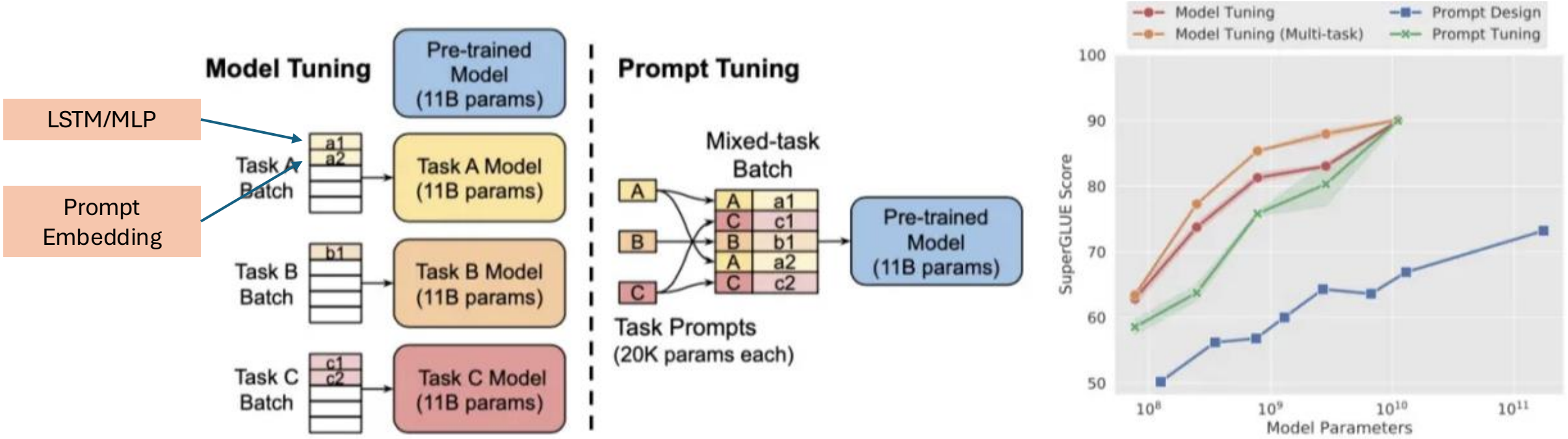


$$X` = [x`_1, x`_2, \ldots, x`_p; x_1, x_2, \ldots, x_m] \qquad Y = WX`$$

W 不变，PLM 不变

1.3.3 p-tuning

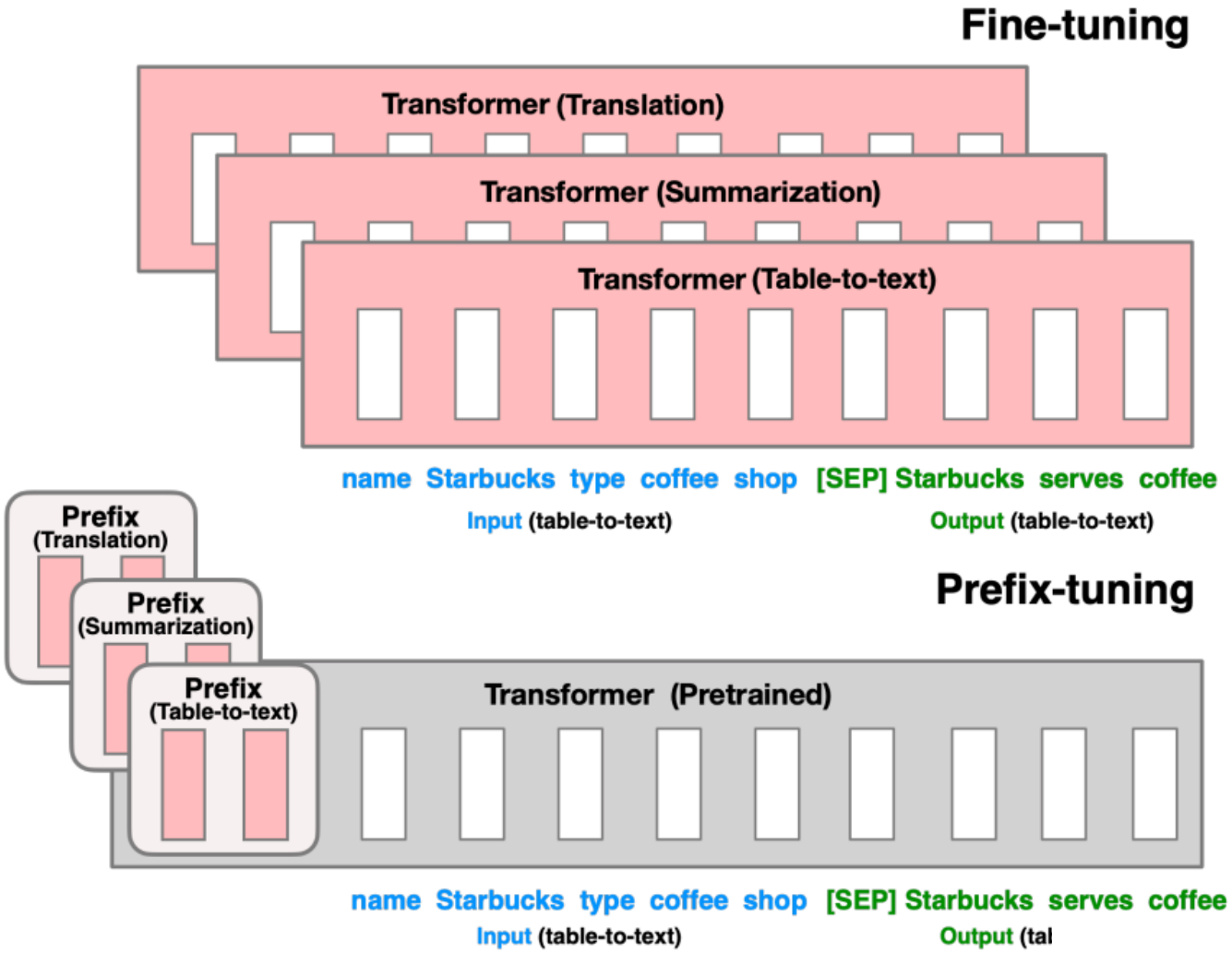
- **P-Tuning的思想**：在Prompt-Tuning的基础上，对Prompt部分进行进一步的编码计算，加速收敛。具体来说，PEFT中支持两种编码方式，一种是LSTM，一种是MLP。与Prompt-Tuning不同的是，Prompt的形式只有Soft Prompt。



$$X` = [x`_1, x`_2, \ldots, x`_p; x_1, x_2, \ldots, x_m] \qquad Y = WX`$$

W 不变，PLM 不变

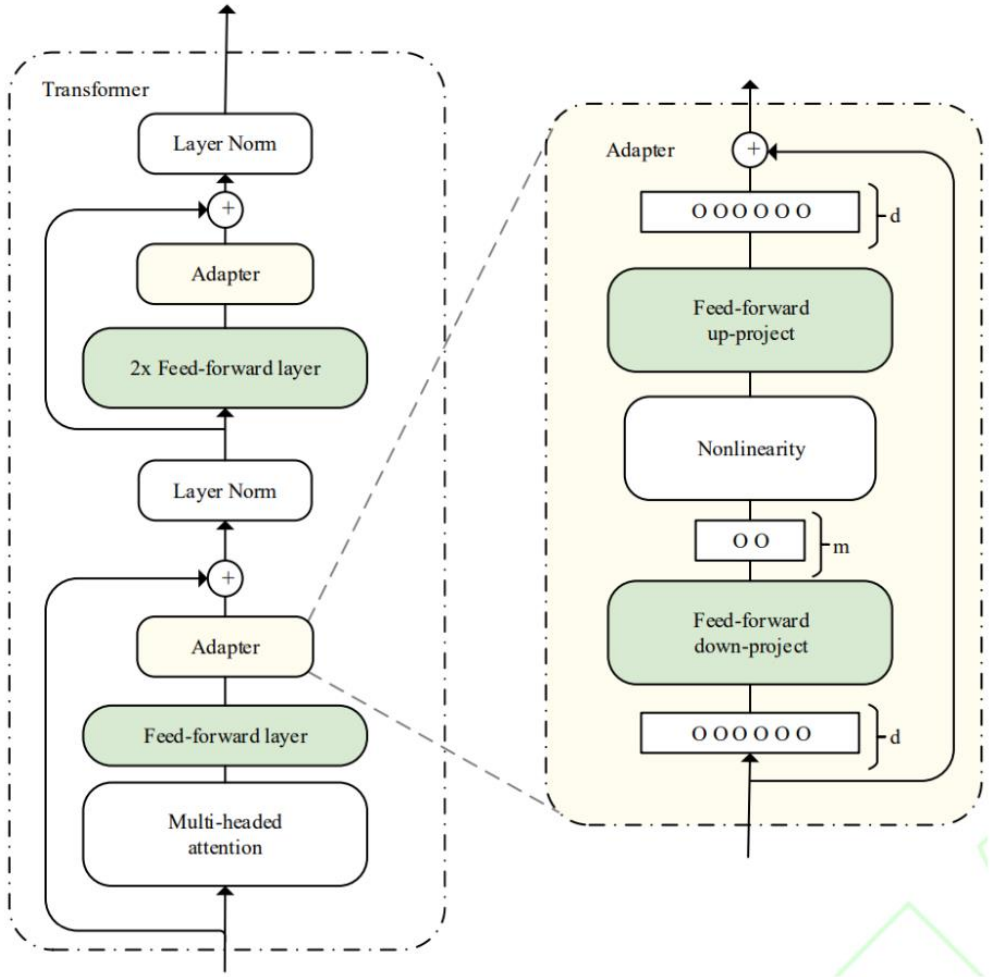
1.3.4 prefix-tuning



past_key_values: Transformer模型中历史计算的key和value的结果，最早是用于生成类模型解码加速，解码过程是根据历史输入，每次预测一个新的token，然后将新的token加输入，再预测下一个token。这个过程中，会存在大量的重复计算，因此可以将key和value的计算结果缓存，作为past_key_values输入到下一次的计算中，这一技术又被称之为kv_cache。

Prefix-Tuning中，就是通过past_key_values的形式将学习的部分放到了模型中的每一层，这部分内容又被称为前缀。

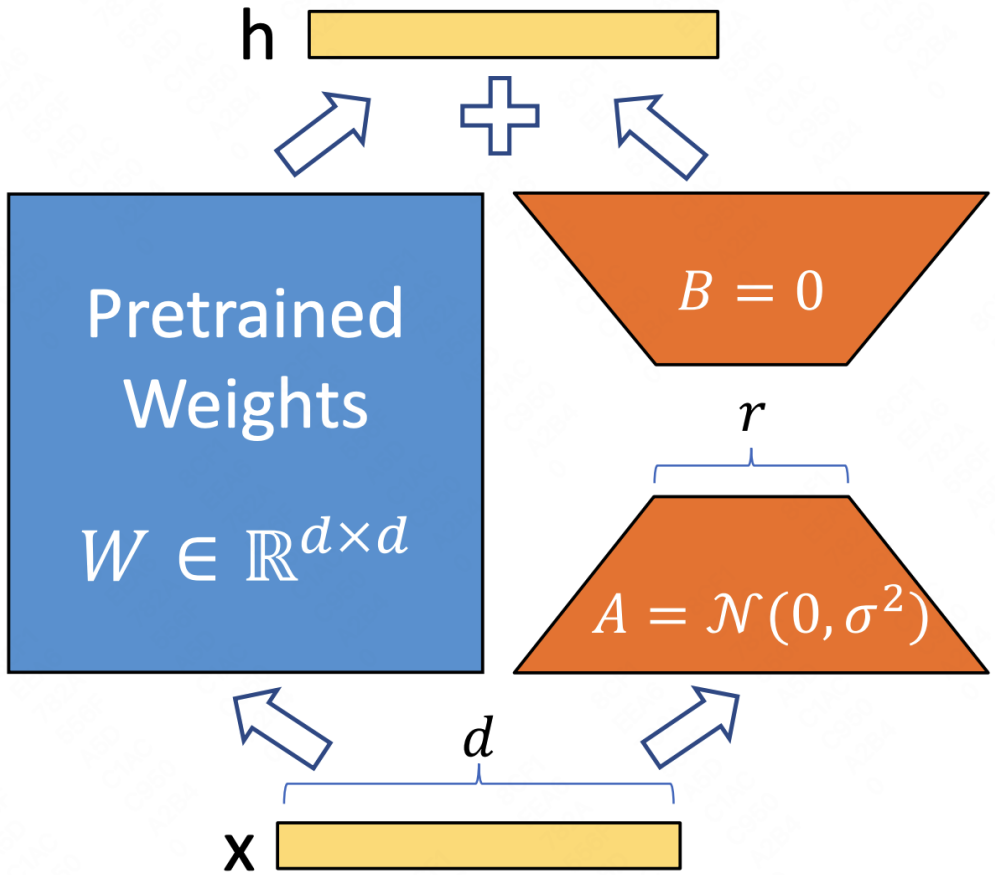
1.3.5 adapter



Adapter的核心思想是在神经网络模块基础上添加一些残差模块，并只优化这些残差模块，由于残差模块的参数较少，因此微调成本更低。

Transformer的**注意力层和前馈神经网络（FFN）层之后添加全连接网络**。微调时，只对新增的Adapter结构和Layer Norm层进行微调，从而保证了训练的高效性。每当出现新的下游任务，通过添加Adapter模块产生一个易于扩展的下游模型，**从而避免全量微调与灾难性遗忘的问题**。

1.3.6 LoRA



假设这是一个4*5的矩阵

$$W = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix}$$

$$\Delta W = A \times B$$

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \\ a_{41} & a_{42} \end{bmatrix}, \quad B = \begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} & b_{15} \\ b_{21} & b_{22} & b_{23} & b_{24} & b_{25} \end{bmatrix}$$

则矩阵乘积 $A \times B$ 将是一个 4×5 的矩阵：

$$A \times B = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} & \cdots & a_{11}b_{15} + a_{12}b_{25} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} & \cdots & a_{21}b_{15} + a_{22}b_{25} \\ \vdots & \vdots & \ddots & \vdots \\ a_{41}b_{11} + a_{42}b_{21} & a_{41}b_{12} + a_{42}b_{22} & \cdots & a_{41}b_{15} + a_{42}b_{25} \end{bmatrix}$$

1.4 怎么才能调好呢

业界共识

1. prompt的质量和多样性远重要于数据量级，**微调一个30b量级的base model只需要10w量级的数据即可。**
参考：《LIMA: Less Is More for Alignment》

2. 合成数据很重要！——一般需要通过不同方式进行多路合成，减少合成数据的bias。
参考：《**Phi-3** Technical Report: A Highly Capable Language Model Locally on Your Phone》

3. 可以加点预训练的数据进去，减少灾难性遗忘现象。
参考：《The Llama 3 Herd of Models》

4. 一般训练一个epoch，垂直领域模型数据少的话训练3epoch去过拟合。

5. 可以全量微调，就不要做PEFT。

6. SFT阶段不能太多知识注入，过多的知识注入，或者超出模型能力本身的回答过多会导致对齐税。

1.4 怎么才能调好呢

数据合成

- 合成Prompt

1. 划分技能库，给每一个数据打上tasktype，越细越好
2. 随机采样seed，结合第一点，从不同数据库中抽取seed（可以分层抽样），造新prompt合成数据
3. 纯指令生成prompt，用启发式的规则过滤
4. 合成不同格式的（如markdown，json），行文风格的prompt 合成Answer

- 指令参考

1.英文数据，GPT4 is all you need
2. 中文模型，Qwen_72B / deepseek_MOE
3. 给定约束条件
4.尽量给CoT

工业界做法
Train一个reward model或者用规则拒绝采样，选出合适的SFT/DPO训练数据

2

显存计算和量化

2.1 显存计算

0.基础知识

1GB = 1024MB

1MB = 1024KB

1KB = 1024Byte

1Byte = 8Bit

FP32 32Bits， 4Bytes

FP16 16Bits， 2Bytes

2.1 显存计算

1.输入输出

Llama13B

b Batch Size : 1

s Sequence Len : 1024

h Hidden Size: 5120

Embedding后输入：

$$b * s * h * 2/1024/1024$$

FP16: 10MB

输入输出： 20MB

2.模型参数

Llama13B

1B=1000^3

1GB=$1024^3 byte$

FP16：13*2 = 26GB

3.优化器

Llama13B

Adam

梯度指数平滑值：13*4 = 52GB

梯度平方指数平滑值：13*4 = 52GB

模型参数：13*4 = 52GB

一共156GB

为什么优化器这里都是FP32?

2.1 显存计算

AdamW

$$g_w = \frac{\partial L}{\partial w}$$

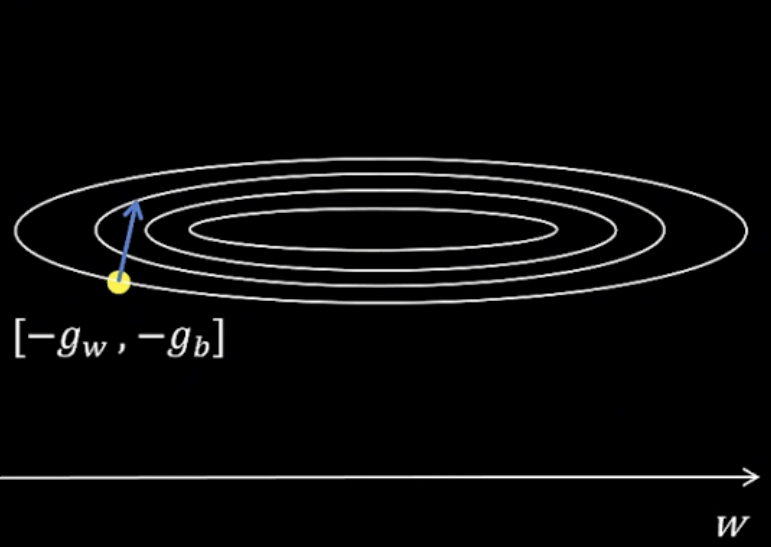
$$V_w = \beta_1 V_w + (1-\beta_1) g_w \quad \beta_1 = 0.9$$

$$S_w = \beta_2 S_w + (1-\beta_2) {g_w}^2 \quad \beta_2 = 0.999$$

$$V_w^{correct} = \frac{V_w}{1-{\beta_1}^t}$$

$$S_w^{correct} = \frac{S_w}{1-{\beta_2}^t}$$

$$w_{t+1} = w_t - r\frac{V_w^{correct}}{\sqrt{S_w^{correct}} + \varepsilon} - r\lambda w_t$$



2.1 显存计算

Adam & AdamW

$$V_w = \beta_1 V_w + (1-\beta_1) g_w \quad \beta_1 = 0.9$$

$$S_w = \beta_2 S_w + (1-\beta_2) {g_w}^2 \quad \beta_2 = 0.999$$

每个参数都要额外保存两个值。

每个值都需要用float32位来存储。

如果参数用float16存储，这两个值占用大小将是参数大小的4倍。

4.激活值

$$x * w_1 = z_1$$

$$sigmoid(z_1) = a_1$$

$$a_1 * w_2 = z_2$$

$$loss = (y - z_2)^2$$

$$\frac{\partial loss}{\partial w_2} = \frac{\partial loss}{\partial z_2} * \frac{\partial z_2}{\partial w_2}$$

$$= 2(z_2 - y) * a_1$$

$$\frac{\partial loss}{\partial w_1} = \frac{\partial loss}{\partial z_2} * \frac{\partial z_2}{\partial a_1} * \frac{\partial a_1}{\partial z_1} * \frac{\partial z_1}{\partial w_1}$$

$$= 2(z_2 - y) * w_2 * \sigma(z_1) * (1 - \sigma(z_1)) * x$$

σ(x)=sigmoid(x)

4.激活值

Llama13B

s 序列长度：1024

b batch size：1

h 隐藏层大小：5120

a Attention头：40

L 层数：40

FP16:

$s * b * h * (34 + 5 * a * s/h) * L/1024/1024/1024$ GB

=14.5GB

5.梯度值

Llama13B

FP16:

$13B * 2 = 26G$

1.1 定义

Llama13B

BatchSize=1， Seq_Len=1024， FP16

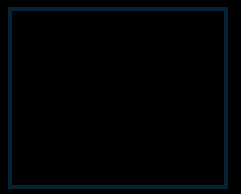
模型参数：26GB

优化器：156GB

激活值：14.5G （和BatchSize，Seq_len相关）

梯度值：26GB

合计：222.5GB



1.2　那么如何降下来呢?

如何降低显存占用?

1.2那么如何降下来呢?

Llama13B

BatchSize=1， Seq_Len=1024， FP16

模型参数：26GB

优化器：156GB

激活值：14.5G （和BatchSize，Seq_len相关）

梯度值：26GB

合计：222.5GB

